CampusFlow Project Context

CampusFlow / CampusNexus Backend — Full Project Context

> This document is a complete, self-contained snapshot of the CampusFlow backend codebase, written so it can be pasted or uploaded into a conversation (e.g. Claude web) that has no direct access to the repository. It was generated by exploring the actual code as of **2026-07-10**. If you're reading this in a future conversation and something seems off, treat it as a snapshot in time, not a live source of truth.

---

1. Product Overview

CampusNexus (internal codebase/package name: **CampusFlow**) is a multi-tenant **SaaS College/Campus ERP** built by **Polynexus Technologies Private Limited** (Nagpur, India) for Indian higher-education institutions. It's sold B2B on a subscription model (trial → monthly/annual billing cycle) with **modular pricing** — colleges subscribe only to the modules they want, and both platform-level and per-role access to those modules is independently gated.

Core product pillars: 1. **Biometric attendance** — face-recognition (InsightFace/ArcFace) attendance with a serious anti-spoofing/liveness stack and a self-improving adaptive template pipeline. 2. **Real-time bus/transit tracking** — WebSocket-driven live GPS tracking, geofenced stop alerts, QR-code boarding passes. 3. **Core academic/ops** — departments, courses, classrooms, lectures/timetables, exams, assignments, leave, payroll, announcements, analytics dashboards. 4. **Finance** — student fee invoicing/online payments (Razorpay) + fully separate B2B offline (NEFT/RTGS) billing that Polynexus charges colleges. 5. **"Competitive parity" modules** — Hostel, Training & Placement (TPO), Library, Inventory, Digital Answer-Sheet Valuation. 6. **Compliance & governance** — full DPDP Act 2023 (India's data-protection law) compliance: biometric/data consent, guardian consent for minors, right-to-erasure, PII masking, plus a global automatic audit-log system. 7. **Multi-tenancy** — one PostgreSQL schema per college via `django-tenants`, with a `public` schema for the SaaS Admin layer that manages tenant provisioning/billing, and a shared public **"demo" tenant** (destructive actions blocked) for sales demos.

Tech stack one-liner: Django 5.1.7 + Django REST Framework, PostgreSQL (schema-per-tenant via `django-tenants`), Django Channels + Redis for WebSockets, JWT auth (`simplejwt`), InsightFace/ArcFace for biometrics, Razorpay for payments, served via `uvicorn` (ASGI) behind Docker/docker-compose.

---

2. Tech Stack & Dependencies

No `pyproject.toml` / `Pipfile` — plain `requirements.txt`. Full contents, annotated:

```
# Django Core
Django==5.1.7
asgiref==3.11.1

# Django REST Framework
djangorestframework==3.15.2
djangorestframework_simplejwt==5.5.0
drf-yasg==1.21.8              # Swagger/OpenAPI docs
django-filter==25.1
django-cors-headers==4.6.0
uritemplate==4.2.0

# Multi-Tenancy
django-tenants==3.7.0

# Authentication & Security
PyJWT==2.9.0
cryptography>=43.0            # Fernet field encryption (payment gateway secrets)

# Payment Gateways
razorpay>=1.4.2

# Database
psycopg-binary==3.2.6
psycopg2-binary==2.9.12
sqlparse==0.5.5

# Caching
django-redis==5.4.0
redis==5.0.1

# WebSockets (Django Channels)
channels==4.2.2
channels_redis==4.2.1
uvicorn[standard]==0.50.0     # ASGI server used in production entrypoint

# Email
django-anymail[brevo]         # transactional email via Brevo API

# Static Files
whitenoise==6.9.0

# Biometrics & Face Recognition
insightface>=0.7              # ArcFace face recognition
onnxruntime>=1.16
numpy>=1.24
opencv-python-headless

# Media & QR
pillow==12.2.0
qrcode==8.2                   # bus route QR codes
```

```
# User Agent Parsing & Client Info
django-ipware==3.0.0
user-agents==2.2.0
ua-parser==1.0.2
ua-parser-builtins==202605

# Utilities
python-dotenv==1.0.1
python-dateutil==2.9.0.post0
pytz==2026.1.post1
tzdata==2026.2
certifi==2026.4.22
urllib3==2.6.3
packaging==26.2
typing_extensions==4.15.0
PyYAML==6.0.3
inflection==0.5.1
```

`package-lock.json` exists at the project root but is essentially empty (`"packages": {}`) — vestigial, no real Node/JS dependency tree.

---

3. Repository Layout

```
CampusFlow_backend/                          (git repo root)
├── README.md                                (essentially empty — just "###")
├── .vscode/launch.json                      (Django debug config: runserver 0.0.0.0:8000)
├── agreement_documents/                      (PDF business docs: trial & subscription agreements)
└── campusflow/                               (Django project root — everything lives here)
    ├── manage.py
    ├── requirements.txt
    ├── Dockerfile
    ├── docker-compose.yml
    ├── entrypoint.sh
    ├── .env / .env.example / .gitignore
    ├── campusflow_logo.svg / campusgrid_logo.svg / campusnexus_logo.svg
    ├── campusflow_postman_collection.json    (also duplicated in Docs/)
    ├── migrate_sync.py                       (restores clean migration files in Docker — see §14)
    ├── fix_timetable.py, locustfile.py       (load testing with Locust)
    ├── seed_demo_data.py, seed_dummy_data.py, seed_extended_demo_data.py,
    │   seed_large_users.py, seed_test_users.py, seed_valuation_data.py  (standalone data seeders)
    ├── test_channel_layer.py, test_consumer.py, test_stops.py, test_ws_connection.py
    │   (ad hoc manual WebSocket test scripts — NOT part of Django's test suite)
    ├── billing_receipts/                     (uploaded FileField media for Invoice.bank_receipt)
    ├── models/insightface/                   (heavy ML model weights, buffalo_l — gitignored)
    ├── templates/admin/ , templates/emails/  (project-level template overrides/email templates)
    ├── Docs/                                  (real project documentation — see §14)
    │
    ├── campusflow/                            (Django project package — settings/urls/asgi/wsgi)
    │   ├── settings.py, urls.py, wsgi.py, asgi.py
    │   ├── middleware.py                     (CampusFlowTenantMiddleware)
    │   └── authentication.py                 (older/possibly-unused duplicate — see §16)
    │
    ├── campusflow_app/                        (the main Django app — nearly all business logic)
    │   ├── admin.py, apps.py, urls.py, serializers.py (951 lines), signals.py (226 lines)
    │   ├── authentication.py                 (TenantAwareJWTAuthentication — the one actually
used)
    │   ├── permissions.py                    (RBAC permission classes)
    │   ├── demo_guard.py                     (public demo-tenant guardrails)
    │   ├── email_backend.py                  (DemoAwareEmailBackend)
    │   ├── payment_gateways.py               (Razorpay/Cashfree/PayU adapter layer)
    │   ├── face_utils.py                     (InsightFace/ArcFace biometric logic)
    │   ├── fields.py                         (EncryptedCharField — Fernet encryption)
    │   ├── tests.py
    │   ├── models/                           (31 files — one per domain, see §6)
    │   ├── views/                            (29 files — one per domain, see §7)
    │   ├── consumers/bus_tracking.py         (Django Channels WebSocket consumer)
    │   ├── middleware/audit.py               (AuditMiddleware — thread-local request capture)
    │   ├── management/commands/              (clear_old_logs, fine_tune_face_embeddings,
    │   │                                       promote_fine_tuned_embeddings, seed_demo_data)
    │   ├── utils/ (file_validation.py, tenant_utils.py) + a separate top-level utils.py
    │   └── migrations/                       (10-12 migration files — gitignored, see §16)
    │
    └── tenants/                               (shared/public-schema app for multi-tenancy)
```

```
        ├── models.py                          (Tenant, Invoice, Domain)
        ├── admin.py, apps.py, views.py (270 lines), serializers.py (233 lines), tests.py
        ├── management/commands/              (create_tenant, fix_domains, run_billing_cron)
        └── migrations/
```

---

4. Multi-Tenancy Architecture

- Built on **`django-tenants`**: one PostgreSQL **schema per college** (`TENANT_MODEL = "tenants.Tenant"`, `TENANT_DOMAIN_MODEL = "tenants.Domain"`).
- `SHARED_APPS` (public schema only): `django_tenants` (must be first), `tenants`, Django contrib apps, `rest_framework`, `rest_framework_simplejwt` (+ token_blacklist), `corsheaders`, `drf_yasg`, `channels`.
- `TENANT_APPS` (replicated per college schema): contenttypes, auth, admin, `rest_framework_simplejwt.token_blacklist`, `drf_yasg`, **`campusflow_app`**.
- `DATABASE_ROUTERS = ('django_tenants.routers.TenantSyncRouter',)`.
- **Tenant resolution** happens in `campusflow.middleware.CampusFlowTenantMiddleware` (must run first in `MIDDLEWARE`), in this priority order: `X-Tenant` header → standard domain-based resolution → JWT payload's `tenant_schema` claim → fallback to `public`.
- A second layer of tenant safety lives in `campusflow_app/authentication.py`'s **`TenantAwareJWTAuthentication`**, which extends simplejwt's `JWTAuthentication` to: (1) decode the JWT's *unverified* `tenant_schema` claim and switch the DB connection to that schema **before** verifying the token (needed because mobile/IP-based requests don't always match a domain), (2) after verification, hard-check the token's signed `tenant_schema` claim matches the now-active schema (prevents replaying a token from one tenant against another via a spoofed `X-Tenant` header), (3) blocks `DELETE` requests entirely when the active tenant `is_demo`.
- **Public "demo" tenant**: `Tenant.is_demo` flag gates a shared public sandbox account used for sales demos. Guardrails live in `campusflow_app/demo_guard.py` (`is_demo_tenant()`, `IsNotDemoTenant` permission, `demo_block_response()`), `email_backend.py` (`DemoAwareEmailBackend` silently drops outgoing mail for the demo tenant so visitors can't spam real inboxes), and the DELETE-block in `TenantAwareJWTAuthentication`.

---

5. Settings Deep-Dive (`campusflow/campusflow/settings.py`, Django 5.1.7)

Security/env loading - `.env` loaded via `python-dotenv` if present. - `SECRET_KEY` and `FIELD_ENCRYPTION_KEY` (Fernet key for encrypting payment-gateway secrets at rest) **must** be set via env when `DEBUG=False`, else the app raises at import time. `SECRET_KEY` also signs JWTs. - `ALLOWED_HOSTS` defaults to `campusnexus.in,.campusnexus.in`; `*` appended in DEBUG. - `CSRF_TRUSTED_ORIGINS` = `https://campusnexus.in`, `https://*.campusnexus.in`. - CORS: `CORS_ALLOW_ALL_ORIGINS = False`; explicit `CORS_ALLOWED_ORIGIN_REGEXES` for `*.campusnexus.in` (+ localhost regex in DEBUG), extendable via `CORS_ALLOWED_ORIGINS` env var. `CORS_ALLOW_HEADERS` extends defaults with `x-tenant` (frontend sends `X-Tenant` for schema routing). - Cookie/transport security (`SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_SSL_REDIRECT`, HSTS 1yr+subdomains+preload) all toggle off in DEBUG. `CSRF_COOKIE_HTTPONLY = False` intentionally (frontend JS needs to read it).

Middleware order (matters):

```
MIDDLEWARE = [
    'campusflow.middleware.CampusFlowTenantMiddleware',  # MUST be first
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    "whitenoise.middleware.WhiteNoiseMiddleware",
    'campusflow_app.middleware.audit.AuditMiddleware',
]
```

REST_FRAMEWORK: auth classes tried in order — `TenantAwareJWTAuthentication` → simplejwt `JWTAuthentication` → `SessionAuthentication` → `BasicAuthentication`. Default permission: `IsAuthenticated`.

Database: PostgreSQL only, via `django_tenants.postgresql_backend`, config from env (`POSTGRES_DB`, `DB_HOST`, `DB_PORT`, `POSTGRES_USER`, `POSTGRES_PASSWORD`), `CONN_MAX_AGE` defaults 60s.

Caching: Redis (`django_redis`) if `REDIS_URL` env set, else Django's `LocMemCache`. Used for OTP/temporary data.

Django Channels (WebSockets): `ASGI_APPLICATION = "campusflow.asgi.application"`; channel layer backend `channels_redis.core.RedisChannelLayer` pointed at `REDIS_URL` (default `redis://127.0.0.1:6379/0`). Powers real-time bus tracking.

Static files: `whitenoise` + `STATIC_ROOT = staticfiles/`.

JWT (`SIMPLE_JWT`): access token 8h, refresh token 7d, `ROTATE_REFRESH_TOKENS=True`, `BLACKLIST_AFTER_ROTATION=True`, `UPDATE_LAST_LOGIN=False` (session invalidation managed via a custom "salt" mechanism instead), HS256 signed with `SECRET_KEY`.

Email: SMTP backend if `EMAIL_HOST` env set, else falls back to `anymail.backends.brevo.EmailBackend` (Brevo/Sendinblue transactional API via `BREVO_API_KEY`). Both wrapped by `DemoAwareEmailBackend`. `CONTACT_RECIPIENT_EMAIL`, `CONTACT_CC_LIST` (`sales@polynexus.in`), `CONTACT_BCC_LIST` (`admin@polynexus.in`) configure the landing-page contact form.

Face recognition / biometrics config block: - `FACE_SIMILARITY_THRESHOLD = 0.55`, `INSIGHTFACE_MODEL_NAME = "buffalo_l"`, `INSIGHTFACE_MODEL_ROOT` under `BASE_DIR/models/insightface`. - `LIVENESS_BLINK_THRESHOLD = 5.5`. - Adaptive learning: `FACE_ADAPTIVE_LEARNING_ENABLED`, threshold `0.75`, max alpha `0.15`; monthly fine-tune: `FACE_MONTHLY_FINE_TUNE_MIN_SAMPLES=6`, outlier floor `0.50`, blend weight `0.4`. Staged results written as JSON under `FACE_FINE_TUNED_OUTPUT_ROOT` for manual review before promotion.

Commented-out/dead code: GDAL/GEOS library paths, a `local_settings.py` import block — both inactive.

---

6. Data Model (by domain)

Models live in `campusflow_app/models/` (31 files, registered via `models/__init__.py`). **10 migrations** exist as of this snapshot (see §6.13 for the migration-by-migration history). No custom model managers — business logic lives in model methods/properties and signal handlers.

6.1 Users / Auth / Org Structure

- **`Department`**: `name`, `code` (unique, regex-validated), `hod` → FK `User` (SET_NULL), contact/accreditation fields, `status`, audit (`created_by` / `updated_by` /timestamps). Organizational anchor — nearly everything else references it.
- **`Course`**: `course_code` (unique), `course_name`, `department` FK (CASCADE).
- **`Classroom`**: `name`, `code` (unique, nullable). Has commented-out GIS `PolygonField` /bounding-box geofence fields — planned but disabled (avoids a GDAL dependency).
- **`Lecture`**: `name`, `subject`, `faculty` FK `User`, `classroom` FK, `start_time` / `end_time`, `code` (unique), `latitude` / `longitude` (geofence anchor).
- **`Schedule`**: `course` FK, `faculty` FK (`related_name='scheduled_classes'`), `classroom` FK, `day_of_week`, `start_time` / `end_time`, `semester`, `academic_year`, `substitute_faculty` FK (SET_NULL). `unique_together=(course, classroom, day_of_week, start_time)` prevents room double-booking.
- **`Location`** (defined in `models/location.py` but **not exported** from `models/__init__.py` or registered in admin — orphaned/legacy model): `location_id`, `name`, lat/lng, `geofence_radius_meters`, `is_premises_entry` / `is_classroom_entry` flags. Intended for QR-code premises entry.
- **`models/profile.py`** (largest model file, ~24KB) — six role-profile models, each `OneToOneField(User, CASCADE)`:
  1. **`StudentProfile`** (`related_name='student_profile'`): `student_id` (unique), `department` FK, `locked_device_id` (single-device lock for face attendance), `is_face_registered`. Extensive personal/academic/address fields, `aadhaar_number` (unique per-model), `biometric_id` (unique). **DPDP block**: `consent_given`, `consent_timestamp`, `consent_version`, plus guardian-consent fields (`parent_guardian_name`, `parent_guardian_email`, `guardian_consent_given`).
  2. **`TeachingStaffProfile`** (`related_name='teaching_staff_profile'`): `employee_id` (unique), professional fields (designation, qualifications, PAN, EPF/ESI), `staff_role` default `'lecturer'`. Same DPDP trio (no guardian fields).
  3. **`NonTeachingStaffProfile`** — `staff_role` default `'administrator'`, DPDP trio.
  4. **`ManagementProfile`** — `staff_role` default `'director'`, DPDP trio.
  5. **`AdministratorProfile`** — `staff_role` default `'system_admin'`, DPDP trio.
  6. **`DepartmentHeadProfile`** — `department` is a **OneToOneField** (enforces one HOD per department), `staff_role` default `'HOD'`, DPDP trio.
     - Quirk: `aadhaar_number` / `pan_number` uniqueness is enforced **per-profile-model**, not globally across all profile tables.

6.2 Attendance / Face Recognition (Biometric)

- **`Attendance`**: `user` FK, `schedule` / `lecture` FK (SET_NULL), `check_in_time` / `check_out_time`, `is_geofence_valid`, `device_id`, `verification_method` (`face_geofence` default / `manual`). `unique_together=(user, lecture)`.

- **`FaceEmbedding`**: `student` FK `StudentProfile` (`related_name='face_embeddings'`), `angle` (front/left/right), `image`, `embedding` (JSON, 512-d ArcFace vector), `sample_count`, `updated_at`. `db_table='face_embeddings'`, `UniqueConstraint(student, angle)`.
- **`FaceEmbeddingSample`**: append-only log of high-confidence live captures for adaptive learning — `student`, `angle`, `embedding`, `confidence_score`, `captured_at`, `staged_at` (set by monthly fine-tune command), `consumed_at` (set by promotion command).
- **`BiometricConsentLog`**: `student` FK, `consent_given`, `consent_timestamp`, `consent_version`, `ip_address`, `user_agent`. **Defined in `face_embedding.py` but not exported from `models/__init__.py`** (only `FaceEmbedding` / `FaceEmbeddingSample` are) — likely an oversight, though its migration (0009) does create the table.
- **`FaceAttendanceLog`**: `student` FK, `lecture` FK, `timestamp`, `confidence_score`, `is_verified`, `liveness_passed`. `db_table='face_attendance_logs'`, `UniqueConstraint(student, lecture)`.
- **`AttendanceSession`**: `lecture` → **OneToOneField** (`related_name='active_session'`), `started_by` FK, `started_at`, `duration_minutes` (default 5), `extended_minutes`, geofence center lat/lng, `radius_meters` (default 100), `tolerance_meters` (default 15), `is_active`.
- **`FraudAlert`**: `student` FK, `lecture` FK, `attempted_at`, `reason` (e.g. "Failed Liveness", "Spoof Attempt"), `confidence_score`, `captured_image` (evidence).
- **`DeviceResetRequest`**: `student` FK, `requested_at`, `status` (pending/approved/rejected), `reason`, `reviewed_by` FK, `reviewed_at`. Governs unlocking `StudentProfile.locked_device_id`.
- **`ManualAttendanceRequest`**: `student` FK, `lecture` FK, `requested_at`, `reason`, `status`, `reviewed_by` FK, `reviewed_at`. `db_table='manual_attendance_requests'`, `UniqueConstraint(student, lecture)`.

6.3 Bus Tracking

- **`BusRoute`**: `name`, `driver` / `conductor` FK `User` (SET_NULL), `stops` (JSON list of `{name, lat, lng}`), `qr_token` (UUID, unique, `regenerate_qr_token()` method invalidates old printed QR codes), `is_active`.
- **`BusSubscription`**: `user`, `route`, `status` (active/expired/suspended), `valid_from` / `valid_until`, `boarding_stop`. `unique_together=(user, route)`. `is_valid` property checks status + date window.
- **`BusAttendance`**: `user`, `route`, `scanned_at`, `device_id` (anti-proxy-scan fingerprint). Indexed on `(user, route, scanned_at)`.
- **`BusLocation`**: `user` → **OneToOneField** (one live-location row per driver, upserted), `lat` / `lng`, `route` (SET_NULL), `updated_at` (auto_now).
- **`BusTrip`**: `driver`, `route` (SET_NULL), `started_at` / `ended_at`, `distance_km`.
- **`BusTrail`**: breadcrumb GPS history — `user`, `route` (SET_NULL), `lat` / `lng`, `timestamp`; ordered/indexed for route replay.

6.4 Fees / Payments / Billing (student-facing)

- **`FeeCategory`**: `name` (unique), `description`.
- **`FeeStructure`**: `name`, `department` FK (SET_NULL), `batch_academic_year`, `program_enrolled_in`, `current_semester_year`.
- **`FeeStructureItem`**: `fee_structure`, `category`, `amount`. `unique_together=(fee_structure, category)`.
- **`StudentFeeInvoice`**: `student` FK `User`, `fee_structure` (SET_NULL), `invoice_number` (unique, auto-generated `INV-<timestamp>-<uuid6>`), `due_date`, `total_amount`, `discount_amount`, `paid_amount`, `status` (unpaid/partially_paid/paid). `remaining_balance` property; `update_payment_status()` method recomputes from related `FeePayment`s.

- **`StudentFeeInvoiceItem`**: `invoice`, `category`, `amount`. `unique_together=(invoice, category)`.
- **`FeePayment`**: `invoice`, `amount_paid`, `payment_method` (cash/upi/card/net_banking/online), `transaction_reference`, `receipt_number` (unique, auto-generated `REC-<date>-<uuid6>`), `payment_date`, `collected_by` FK (SET_NULL), `gateway_transaction` → **OneToOneField** to `PaymentGatewayTransaction` (SET_NULL). `save()` also calls `invoice.update_payment_status()`.
- **`PaymentGatewayTransaction`**: `invoice` FK (CASCADE), `initiated_by` FK (SET_NULL), `gateway` (razorpay/cashfree/payu/phonepe/paytm/mobikwik), `gateway_order_id`, `gateway_payment_id`, `amount`, `convenience_fee_amount`, `total_amount`, `currency` (default INR), `status` (created/paid/failed/refunded), `raw_create_response` / `raw_webhook_payload` (JSON audit trail), `failure_reason`.

6.5 Payroll / HR

- **`SalaryStructure`**: `user` → OneToOneField. Earnings (`basic_pay`, `hra`, `da`, `ta`, `other_allowances`), deductions (`pf_deduction`, `esi_deduction`, `tds_deduction`, `other_deductions`). Computed properties: `gross_salary`, `total_deductions`, `net_salary`.
- **`Payslip`**: `user` FK, `month` / `year`, attendance snapshot, pay breakdown snapshot, `status` (draft/finalized/paid), `finalized_by` FK. `unique_together=(user, month, year)`.
- **`LeaveType`**: `name`, `code` (unique), `max_days` (default 12), `is_paid`, `applicable_to` (JSON role list), `carry_forward`, `is_active`.
- **`LeaveBalance`**: `user`, `leave_type`, `academic_year`, `allocated`, `used`, `carried`. `remaining` property = `allocated + carried - used`. `unique_together=(user, leave_type, academic_year)`.
- **`LeaveRequest`**: `user`, `leave_type`, `start_date` / `end_date`, `reason`, `status` (pending/approved/rejected/cancelled), `approved_by` FK, `rejection_reason`. `num_days` property.

6.6 Academics (Exams, Assignments, Valuation)

- **`ExamType`** (`name`, `code` unique) / **`Exam`**: `exam_type`, `department`, `course`, `date` / `start_time` / `end_time`, `classroom` (SET_NULL), `total_marks` / `passing_marks`, `semester`, `academic_year`, `question_paper_url`, `question_structure` (JSON), `invigilator` FK, `created_by` FK, `status` (scheduled/ongoing/completed/cancelled).
- **`Assignment`**: `title`, `description`, `department`, `course`, `due_date`, `attachment`, `created_by` FK.
- **`AssignmentSubmission`**: `assignment`, `student` FK `User`, `submitted_at`, `attachment`, `text_submission`, `grade`, `feedback`, `status` (submitted/graded). `unique_together=(assignment, student)`.
- **`ValuationSession`**: `exam` FK, `evaluator` FK `TeachingStaffProfile`, `started_at`, `status` (Active/Completed) — blind/scanned-paper evaluation.
- **`ScannedPaper`**: `session`, `student` FK `StudentProfile`, `scanned_file_url`, `allocated_marks`, `evaluated_at`, `status` (Pending/Evaluated), `remarks`, `mask_code` (unique, auto-generated `VAL-<6 random chars>` — enables blind/anonymous evaluation), `question_scores` (JSON).

6.7 Hostel

`Hostel` (name, `gender_type` Boys/Girls/Co-ed, capacity, address), `HostelRoom` (`hostel` FK, `room_number`, `capacity` default 4, `rent_per_semester`, `occupied_beds`; `unique_together=(hostel, room_number)`), `HostelAllocation` (`student` FK, `room` FK, `allocated_date`, `vacated_date`, `status`).

6.8 TPO / Placements

`RecruitmentDrive` (`company_name`, `job_title`, `job_description`, `eligibility_criteria`, `package_lpa`, `drive_date`, `status`), `PlacementApplication` (`drive`, `student` FK, `applied_date`, `status` Applied/Shortlisted/Interviewing/Selected/Rejected; `unique_together=(drive, student)`).

6.9 Library

`Book` (`title`, `author`, `isbn` unique, `publisher`, `total_copies`, `available_copies`), `BookCopy` (`book`, `barcode` unique, `status` Available/Issued/Lost), `BookIssue` (`book_copy`, `student` / `staff_user` FK — nullable either/or, `issued_date`, `due_date`, `returned_date`, `fine_amount`, `status`). Copy counts kept in sync via signals (see §11).

6.10 Inventory

`InventoryCategory` (name, description), `InventoryItem` (`category`, `name`, `quantity`, `unit`, `threshold_level` for low-stock alerts), `Supplier`, `InventoryTransaction` (`item`, `transaction_type` Restock/Issue, `quantity`, `department` FK SET_NULL, `transaction_date`, `supplier` FK SET_NULL, `remarks`).

6.11 Announcements / Module Permissions

- **`Announcement`**: `title`, `content`, `author` FK, `priority` (low/normal/high/urgent), `target_roles` (JSON list), `target_departments` → **ManyToManyField** `Department` (the only M2M in the schema), `is_pinned`, `expires_at`. Ordering `['-is_pinned', '-created_at']`.
- **`TenantModulePermission`**: `group_name` (unique), `allowed_modules` (JSON list) — powers per-tenant, per-role feature-gating.

6.12 Compliance / DPDP / Audit Logging

- DPDP consent fields (`consent_given` / `consent_timestamp` / `consent_version`) are duplicated across all 6 profile models, plus `StudentProfile`'s guardian-consent fields and the dedicated `BiometricConsentLog` model.
- **`AuditLog`**: `user` FK (SET_NULL), `action` (CREATE/UPDATE/DELETE), `model_name`, `object_id`, `object_repr`, `changes` (JSON diff `{field: {old, new}}`), `ip_address`, `user_agent`, `endpoint`, `timestamp` (indexed). Ordered `['-timestamp']`.

6.13 Migrations (chronological, tracks feature evolution)

#	Notable content
0001	Bulk initial schema — Department, Course, Classroom, Lecture, Schedule, all 6 profiles, Attendance, FaceEmbedding, FaceAttendanceLog, AttendanceSession, FraudAlert, DeviceResetRequest
0002	Constraint tweaks to Attendance
0003	AuditLog, Announcement, Leave*, SalaryStructure/Payslip, ExamType/Exam — "new modules" wave (HR + compliance + academics)
0004	Assignment + AssignmentSubmission
0005	ManualAttendanceRequest
0006	Major wave: Fees, TenantModulePermission, BusRoute + related bus models
0007	"Competitive parity" wave: Library, Hostel, Inventory, TPO, Valuation
0008	Adds `conductor` FK to BusRoute
0009	**DPDP compliance wave**: consent fields on all 6 profile models, guardian-consent fields on StudentProfile, creates BiometricConsentLog
0010	(most recent) Adds `sample_count` / `updated_at` to FaceEmbedding; creates FaceEmbeddingSample for adaptive/monthly fine-tuning

6.14 Entity-Relationship Summary (prose)

- `User` (Django auth) is the root identity; every person has exactly one of 6 role-specific profile tables via `OneToOneField`, each optionally scoped to a `Department`.
- `Department` is the organizational anchor: owns `Course`s, HOD-linked to a `User` / `DepartmentHeadProfile`, referenced by `Announcement` (M2M), `Exam`, `Assignment`, `FeeStructure`, `InventoryTransaction`, `Location`.
- **Academic scheduling chain**: `Course` → `Schedule` (faculty + classroom + day/time) → `Lecture` (concrete session) → `Attendance` / `AttendanceSession` / `FaceAttendanceLog` / `FraudAlert` / `ManualAttendanceRequest` all hang off `Lecture` and `StudentProfile`.
- **Biometric pipeline**: `StudentProfile` → `FaceEmbedding` (live template, 1/angle), refined over time via `FaceEmbeddingSample` (staging log), audited via `BiometricConsentLog`; verification attempts logged in `FaceAttendanceLog` (success) / `FraudAlert` (failure/spoof).
- **Financial chain**: `FeeCategory` / `FeeStructure` / `FeeStructureItem` (templates) → `StudentFeeInvoice` / `StudentFeeInvoiceItem` (billed to a `User`) → settled by `FeePayment`, optionally originating from a `PaymentGatewayTransaction` (1:1 link back to the `FeePayment` it produced).
- **Bus tracking**: `BusRoute` (driver+conductor Users) has `BusSubscription`s, `BusAttendance` (QR boarding events), `BusLocation` (live GPS, 1/driver), `BusTrail` (breadcrumbs), `BusTrip` (discrete journeys).
- **HR chain**: `User` → `SalaryStructure` (1:1) feeds `Payslip` (monthly, attendance-linked snapshot); `LeaveType` → `LeaveBalance` / `LeaveRequest` per user per academic year.
- **Academics extended**: `Exam` → `ValuationSession` (evaluator = `TeachingStaffProfile`) → `ScannedPaper` (per-student, blind via `mask_code`); separately `Assignment` → `AssignmentSubmission`.

- **Support modules** (Hostel, Library, Inventory, TPO) each follow a template→instance→transaction pattern rooted in `StudentProfile` or `Department`.
- **Cross-cutting**: `AuditLog` is populated automatically for nearly all model mutations via global Django signals (not per-model FKs); `TenantModulePermission` gates feature access per role-group per tenant schema.

---

7. Views / API Surface

`campusflow_app/views/` has 29 files. `campusflow_app/serializers.py` (951 lines) holds most serializers; some domains (fees, bus_tracking, library, tpo, valuation, inventory, hostel, classroom, face_attendance, users) define their own serializers inline in the corresponding `views/*.py` file instead.

File	Purpose / Endpoints
`users.py` (2343 lines — largest file in the project)	Auth & identity core: JWT login, OTP-based registration (student/staff) with multi-tenant email-domain routing, pre-created student onboarding OTP flow, forgot-password 3-step OTP flow, device-lock reset for face-recognition rebind, per-role profile CRUD, approvals, tenant settings, **DPDP compliance suite** (guardian consent approval, right-to-erasure, consent withdraw/grant).
`face_attendance.py` (25.7KB)	Face registration (3-angle enrollment + consent gate), liveness challenge issuance, `MarkAttendanceView` (core pipeline: liveness → challenge validation → motion liveness → embedding extraction → match → adaptive learning → attendance record), attendance history, manual-request fallback.
`lecturer_attendance.py`	Faculty-side session control: check-in, start-session, status, manual-request review/approval (single + bulk), device-reset approvals, conducted-history.
`attendance.py`	Manual staff marking, filtered attendance listing, QR/code + geofence check-in alternative to face.
`classroom.py`	Classroom CRUD; geofence/polygon point-in-polygon checks are currently **disabled/stubbed** (GDAL dependency commented out, returns 501).
`location.py`	QR check-in point CRUD.
`lecture.py`	Lecture CRUD, lookup by classroom, random attendance-code generation.
`bus_tracking.py` (39.7KB — largest view file)	Route CRUD + branded QR generation/regeneration, subscription management, boarding scan (duplicate-daily-boarding prevention), live locations, trail history, driver dashboard, summary stats, trip start/end + driver stats.
`payments.py`	Online fee payment: create Razorpay order (+ convenience-fee surcharge calc), post-checkout verify, signature-verified idempotent webhook.
`fees.py`	Fee categories/structures/items, invoice ViewSet, bulk invoice generation, manual/offline payment recording, dashboard.
`payroll.py`	Salary structures, single/bulk payslip generation, payslip listing.
`leave.py`	Leave types, balance, request create/list/action, “my leaves”.
`exam.py`, `course.py`, `schedule.py`	Exam types/exams, course CRUD, timetable listing.
`assignment.py` / `submission.py`	Assignment CRUD + submissions + grading.
`announcement.py`	Announcement CRUD.
`audit.py`	Filterable audit-log listing.
`analytics.py`	Dashboard KPIs: overview, attendance trends, department performance, leave analytics, payroll summary.
`module_permissions.py`	Tenant module subscription, role-module permission matrix, custom roles, “my allowed modules”.

File	Purpose / Endpoints
`hostel.py`, `inventory.py`, `library.py`, `tpo.py`, `valuation.py`	DRF `ModelViewSet`s for the "competitive parity" modules.
`contact.py`	Landing-page enquiry emailer.
`department.py`	Department CRUD — gates all other user creation.

URL routing: `campusflow/campusflow/urls.py` mounts `campusflow_app.urls` **both** unprefixed (`''`) and under `api/` — every route is reachable both ways. Also has `admin/`, SaaS tenant/billing routes (`api/saas/...`), and `swagger/` / `redoc/` (drf-yasg). `campusflow_app/urls.py` has ~240 routes across roughly two dozen functional groupings (auth, DPDP/consent, billing, approvals, profiles, department/location/classroom, attendance, face-attendance, lecturer-session, lecture, permissions/employees, audit, announcements, leave, payroll, exams/timetable, analytics, assignments, bus tracking, fees/accounts, online payments, module subscriptions, hostel/tpo/library/inventory/valuation).

---

8. Face Recognition & Liveness Pipeline (`campusflow_app/face_utils.py`)

Library: InsightFace (ArcFace), `buffalo_l` model, CPU inference, lazy singleton loader, `det_size=(320,320)` (tuned for speed on close-up selfies).

- `extract_embedding(image_bytes)` → 512-d L2-normalized embedding. Rejects no-face / picks-largest-if-multi-face / low-confidence (`det_score < 0.5`).
- `extract_embedding_with_pose(...)` → embedding + yaw/pitch/roll in one pass.
- `compare_embeddings(live, stored[], threshold=0.55)` → cosine similarity match decision.
- `blend_embedding(old, new, alpha)` — weighted EMA blend for adaptive template learning.
- **Anti-spoofing / liveness (multi-layer):**
  - `_anti_spoof_score` — "variance-of-variances" texture analysis + FFT peak-ratio analysis (screens imprint a periodic pixel-grid signature); flags spoof only when both signals agree.
  - `basic_liveness_check` — SCRFD detection confidence + anti-spoof texture/FFT check.
  - `detect_specular_flash_reflection` — detects screen-glare from someone replaying a spoof video/photo on another device; masks eye/glasses regions to avoid false positives on glasses-wearers.
  - `check_frame_motion` — **blink-based liveness**: compares two frames' eye-region pixel MAD (real blink ≈ 30-80, static photo ≈ 0-3); also checks a nose-region "control" motion to reject whole-face rigid shifts as false positives; threshold `LIVENESS_BLINK_THRESHOLD` (default 5.5).
  - `check_head_motion` — nod/turn-left/turn-right challenge verification via scale-invariant nose displacement.
  - `check_head_pose` — validates registration-angle photos (front/left/right) using empirically-derived yaw thresholds.
- This powers a **challenge-response liveness system**: server issues a random single-use challenge (blink/nod/turn_left/turn_right) → client captures baseline + action frame → server validates the specific motion happened → then does the face match.

Adaptive learning pipeline (real-time + monthly batch, human-in-the-loop): - During `MarkAttendanceView`, high-confidence live captures are logged to `FaceEmbeddingSample` and the live template is nudged via a bounded EMA blend. - **`fine_tune_face_embeddings` management command** (intended monthly cron): consolidates each

student's accumulated samples into an outlier-rejected centroid candidate per student/angle, blends with the old embedding, and **stages results as a JSON manifest** for human review — never writes directly to the live `FaceEmbedding` table. Supports `--dry-run`; runs per-tenant. - **`promote_fine_tuned_embeddings` management command**: the human-in-the-loop apply step. Takes a reviewed manifest and writes the consolidated embedding into the live `FaceEmbedding` record, but only if the live template hasn't drifted since staging (cosine similarity safety floor of 0.999, skippable via `--force`). Marks consumed samples, archives applied manifests.

---

9. Real-Time Bus Tracking (Django Channels)

- **`campusflow/campusflow/asgi.py`** wraps HTTP with `ProtocolTypeRouter`; WS route `ws/bus-tracking/` → `BusTrackingConsumer`, wrapped in `AuthMiddlewareStack`.
- **`campusflow_app/consumers/bus_tracking.py`** — `BusTrackingConsumer(AsyncWebsocketConsumer)`:
  - **Auth**: JWT `token` query param (decoded via `simplejwt`) + `schema` param for tenant routing; falls back to session auth.
  - **Roles**: drivers/conductors on an active `BusRoute` push GPS; admins/staff join `bus_admins` group and get an `initial_state` snapshot on connect; riders send `{"action":"track_route","route_id":...}` to subscribe to a route's group, gated by an active `BusSubscription`.
  - **GPS ingestion**: filters jitter (<8m ignored), rejects speed outliers (>150km/h teleports), upserts `BusLocation`, appends a `BusTrail` breadcrumb, broadcasts `bus_location_update`, downsamples trail to 120 points for payload size.
  - **Geofencing**: haversine distance to each stop; fires `bus_geofence_alert` on *entering* a 2000m radius of a stop (not every tick).
  - **Mock simulator**: if a route has no live driver connected, auto-generates an interpolated path between stops and streams simulated position updates every 10s (keeps demo/sales maps populated).
  - Uses `channel_layer.group_send` / `group_add` (Redis channel layer) + `database_sync_to_async` + `schema_context` for tenant-safe async DB access.
- QR "boarding pass": `BusRouteQRView` generates a custom branded purple "CampusNexus Transit Boarding Pass" PNG (with logo); `BusRouteRegenQRView` invalidates old printed codes.

---

10. Payments & Billing (two separate systems)

(A) Student-facing online fee payment (`payment_gateways.py`, `models/payments.py`, `views/payments.py`, `views/fees.py`) - Adapter pattern: `BaseGatewayAdapter` interface (`create_order`, `verify_payment_signature`, `verify_webhook_signature`, `parse_webhook_event`). `RazorpayAdapter` is fully implemented; `CashfreeAdapter` / `PayUAdapter` are stubbed (raise `GatewayNotImplementedError`). Each tenant picks one active gateway and stores its own encrypted credentials on the `Tenant` model (`EncryptedCharField`). - `PaymentGatewayTransaction` tracks each checkout attempt (created→paid/failed/refunded), computes optional convenience-fee surcharge (`fee_surcharge_mode`: absorb vs. pass-to-student), links 1:1 to the resulting `FeePayment` once paid. The webhook is the authoritative fallback if the frontend's post-checkout verify call never fires.

(B) Offline B2B billing — Polynexus charges *colleges* for the SaaS subscription itself (`tenants/models.py` , `tenants/views.py` , `run_billing_cron.py`) - `Invoice` model: NEFT/RTGS bank-transfer invoicing, lifecycle `pending_payment → under_review → paid/overdue` , receipt file + UTR number upload. - `InvoiceListAPIView` shows Polynexus's bank account details to college admins; `InvoiceUploadReceiptAPIView` lets them attach proof; `SaaSInvoiceApproveAPIView` / `RejectAPIView` (Polynexus staff only) verify and extend `subscription_end_date` (or bounce it back for re-upload), sending confirmation/rejection emails. - `run_billing_cron` (daily cron): suspends tenants whose trial/subscription has expired; auto-generates the next invoice 7 days before expiry (rate = `billing_student_rate` × active-student-count, min ₹5000 base fee); sends a 3-day-out reminder email.

---

11. Compliance & Governance

Audit logging (`signals.py` , `models/audit.py` , `middleware/audit.py`) - Global Django signal receivers (`pre_save` / `post_save` / `post_delete`) auto-log every CREATE/UPDATE/DELETE on any `campusflow_app` model (except `AuditLog` itself) and `auth.User` — diffing old vs. new values, redacting `password` fields as `[HIDDEN]` , capturing acting user/IP/user-agent/endpoint via thread-local request context. - Throttled self-cleanup purges records older than 30 days at most once/day (cache flag), reinforced by the standalone `clear_old_logs` management command. - Also has dedicated Library signal receivers keeping `Book.total_copies` / `available_copies` in sync with `BookCopy` / `BookIssue` status changes.

DPDP Act 2023 compliance suite (`views/users.py` , `models/profile.py` , `models/face_embedding.py`) - Explicit `biometric_consent_given` gate before any face embedding is stored, logged immutably to `BiometricConsentLog` (IP, user-agent, version, timestamp) via `ipware` . - Minors (computed from DOB) require parent/guardian name+email at registration and sit in `pending_guardian` status until the public `GuardianConsentApprovalView` link is used. - `UserGrantConsentView` / `UserWithdrawConsentView` — Section 6 consent grant/withdraw; withdrawing locks the account (`status='pending'`). - `UserDataErasureView` — Section 12 "Right to Erasure": deletes face-template images/embeddings, nullifies PII fields per role, anonymizes the `User` record, deactivates login, writes an `AuditLog` entry. - Aadhaar numbers masked (`********9012`) in list responses for non-admin viewers. - `demo_guard.py` blocks all mutating/destructive/config actions for every visitor to the public "demo" tenant (shared credentials).

---

12. RBAC & Module Subscriptions

- **Role hierarchy** (`permissions.py`): SaaS Admin (Django superuser, public schema) > Management > Administrator > Department Head > Faculty > Support Staff > Student. Enforced by composable DRF permission classes (`IsSaaSAdmin` , `IsCollegeAdmin` , `IsFacultyOrAbove` , etc.), driven by Django Groups. Department must exist before any other role can be created; only SaaS Admin can create the first College Admin per tenant.
- **SaaS-level**: `TenantSubscriptionView` lets Polynexus assign which premium modules (`bus-tracking` , `hostel` , `tpo` , `library` , `inventory` , `valuation` , `fees` , `payroll` , `exams` , `leave` , `assignments` , `announcements` , `attendance` , etc.) a college has paid for (`Tenant.subscribed_modules`).
- **College-level**: `RoleModulePermissionView` / `MyAllowedModulesView` let college admins further restrict which *subscribed* modules each role (including custom roles via `CustomRolesView`) can see, with premium-module

intersection logic. Primary admin roles (Management/Administrator) are locked from modification by delegated managers.

- Fully documented in `Docs/permissions_reference.md`.

---

13. Management Commands

Command	App	Purpose
`fine_tune_face_embeddings`	campusflow_app	Monthly adaptive face-template consolidation (stages to JSON for review)
`promote_fine_tuned_embeddings`	campusflow_app	Human-reviewed promotion of staged templates into live `FaceEmbedding`
`clear_old_logs`	campusflow_app	Deletes `AuditLog` rows older than 30 days across all tenant schemas
`seed_demo_data`	campusflow_app	Seeds a tenant schema with a realistic demo timetable/department/courses/classrooms/lectures for N past weeks
`create_tenant`	tenants	Provisions a new tenant/college schema
`fix_domains`	tenants	Domain-record repair utility
`run_billing_cron`	tenants	Daily: suspends expired tenants, generates upcoming invoices, sends reminder emails

---

14. Deployment & Docs

Docker/docker-compose (`campusflow/` root): - **Dockerfile**: `python:3.10-slim` base; installs `build-essential`, `libpq-dev`, `postgresql-client`, `libglib2.0-0`, `libgl1` (last two for OpenCV/InsightFace); installs `requirements.txt`; backs up a clean copy of `campusflow_app/migrations` to `/tmp/clean_migrations` (since migrations are gitignored except `__init__.py`); exposes port 8000. - **docker-compose.yml**: `web` (host `8200` →container `8000`, live-mounted repo volume, `.env`, overrides `DB_HOST=db` / `REDIS_URL=redis://redis:6379/0`, runs via `entrypoint.sh`), `db` (`postgres:17`, persistent volume), `redis` (`redis:7-alpine`, persistent volume). - **entrypoint.sh**: waits for Postgres; re-installs `requirements.txt` on every start; runs `migrate_sync.py` (restores clean migrations from the build-time backup); runs `makemigrations`; runs `migrate_schemas --shared` then `migrate_schemas`; auto-creates `public` Tenant + `localhost` Domain if missing; auto-creates a superuser (`admin` / `admin@campusflow.com` / `admin`) if missing; runs `collectstatic`; launches `uvicorn campusflow.asgi:application --host 0.0.0.0 --port 8000 --workers ${WEB_CONCURRENCY:-4}` (ASGI so HTTP + Channels WS both work). - `.env.example` only documents email/Redis/Brevo settings — does **not** document `SECRET_KEY`, `FIELD_ENCRYPTION_KEY`, DB vars, or CORS/ALLOWED_HOSTS vars even though `settings.py` reads them. Treat it as incomplete. - `.gitignore` excludes standard Python/Django artifacts, `.env`, `media/`, `staticfiles/`, `node_modules/`, `.vscode/`, `models/insightface/` (ML weights), `fine_tuned_embeddings/` (staged biometric templates), and **all migration `.py` files except `__init__.py`** — migration history is not committed to git; it's regenerated via `makemigrations`

at container start using `migrate_sync.py` + the Docker-build backup. - No CI config anywhere (no `.github/workflows`, `.gitlab-ci.yml`, `Jenkinsfile`).

Docs/ (the real documentation, `campusflow/Docs/`): - `api_summary.md` — endpoint count summary. - `auth_flow.md` — full auth/tenant-routing walkthrough. - `permissions_reference.md` — complete RBAC endpoint matrix. - `mobile_app_scope.md` (+ PDFs) — mobile app product scope: Student + Faculty + Bus Driver get mobile login (v1); Dept Head gets a lightweight approvals/alerts view (v2); Management/Administrator/SaaS Admin are web-only. - `import_instructions.md`, `walkthrough.md` — Postman collection usage. - `user_login_manual.md` / `CampusNexus_User_Manual.pdf` — end-user manual. - `campusflow_postman_collection.json` / `..._environment.json` — Postman collection/env (`base_url` defaults `http://localhost:8000`).

Load/seed testing: `locustfile.py` (Locust, targets `https://api.campusnexus.in` by default, logs in as `demo_admin`), several standalone `seed_*.py` scripts for populating demo/test tenant schemas.

Tests: `campusflow_app/tests.py` (`DPDPComplianceTests`) covers consent-gated registration, minor/guardian flow, Aadhaar masking, face-registration consent gate, forgot-password OTP flow, profile update. `tenants/tests.py` (`BillingSystemTests`) covers invoice listing, NEFT/RTGS receipt upload, SaaS admin approve/reject, billing-cron suspension. No dedicated automated tests for the bus-tracking WebSocket or face-recognition matching itself (the root-level `test_*.py` WS scripts are manual dev tools, not part of the Django suite).

---

15. Recent Development History (from git log, newest first)

1. **Real-time bus tracking system** using Django Channels, WebSockets, and geofencing (refined driver/admin views, geofence alerts on stop approach).
2. **Live bus-tracking WebSocket consumer, auth views, and API tests** (also added mobile app docs/PDFs, Postman collection, demo_guard, email backend).
3. Merge resolving serializer conflicts.
4. **Payment gateway transactions + automated model audit logging with cleanup** (Razorpay integration, `AuditLog` signals, encrypted credential fields, `clear_old_logs`).
5. **DPDP biometric notice, consent logs, password recovery flow, and offline B2B bank-transfer billing** (guardian consent for minors, `BiometricConsentLog`, forgot-password OTP flow, NEFT/RTGS `Invoice` upload/approval workflow, `run_billing_cron`).
6. User registration & OTP verification, multi-tenant domain validation, role-based registration.
7. Docker-compose (web, postgres, redis) added.
8. Module subscription management + role-based permission control system (`TenantModulePermission`, subscribed-modules matrix).
9. Audit request middleware + container entrypoint DB sync.
10. Attendance tracking with geofence validation, manual overrides, device-locked check-in security.
11. Earlier: assignment management, classroom attendance, leave management, load-testing scripts, contact/enquiry endpoint via Brevo SMTP.

Deeper history shows module-by-module evolution: bus QR boarding → fee/accounts module → Hostel/TPO/Library/Inventory/Valuation ("parity" modules) → dynamic role permission matrix → custom organization roles → branded QR boarding-pass cards → InsightFace model config → real-time bus geofence alerts.

---

16. Known Quirks / Things to Double-Check

These were surfaced during codebase exploration — worth a quick grep/read before relying on them for anything consequential, since they may have been fixed since this snapshot:

1. **Duplicate `authentication.py`** — `campusflow/campusflow/authentication.py` (project-level) appears to be an older/unused duplicate; the one actually wired into `REST_FRAMEWORK` settings is `campusflow_app/authentication.py` (`TenantAwareJWTAuthentication`).
2. **`Location` model orphaned** — defined in `models/location.py` but not exported from `models/__init__.py`, not in admin. Likely superseded by other geofencing logic (`AttendanceSession`, `Lecture` lat/lng).
3. **`BiometricConsentLog` not exported** from `models/__init__.py` despite its migration (0009) creating the table and it being actively written to from views.
4. **Many newer/"parity" modules are not registered in Django admin** — only the original core models (Department, profiles, Course, Schedule, Classroom, Lecture, Attendance, *FaceEmbedding*, FraudAlert, DeviceResetRequest) are. Announcement, Leave, *Payroll, Exam*, Assignment, *Bus*, Fees, *PaymentGatewayTransaction, TenantModulePermission, Hostel*, TPO, *Library*, Inventory, *Valuation*, AuditLog are managed entirely through the REST API.
5. **`.env.example` is incomplete** relative to what `settings.py` actually reads (missing `SECRET_KEY`, `FIELD_ENCRYPTION_KEY`, DB vars, CORS/ALLOWED_HOSTS vars).
6. **Classroom geofence polygon checks are disabled** — GDAL-based `PolygonField`/point-in-polygon logic is commented out in `models/classroom.py` and the corresponding view returns HTTP 501; the project uses simpler lat/lng + radius geofencing elsewhere instead (`AttendanceSession`, `Lecture`).
7. **`aadhaar_number` / `pan_number` uniqueness is per-profile-model**, not enforced globally across all 6 profile tables — theoretically the same Aadhaar number could exist on both a `StudentProfile` and a `TeachingStaffProfile` row.
8. **README.md is essentially empty** (`###`) — this document is intended to be the actual project overview.
9. Migration files (`.py`) are **gitignored** except `__init__.py` — actual schema history isn't in version control; it's reconstructed via `migrate_sync.py` + `makemigrations` at container startup, restoring from a Docker-build-time backup.
10. Product is branded **"CampusNexus"** in user-facing docs/QR codes/emails but the codebase/package/repo is named **CampusFlow** throughout.

End of context document. Generated by exploring the repository at `d:\Polynexus\Servers\Campusnexus\Backend\CampusFlow_backend` on 2026-07-10.